



Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
«Московский государственный технический университет
имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ

ИУ «ИНФОРМАТИКА И СИСТЕМЫ УПРАВЛЕНИЯ»

КАФЕДРА

ИУ7 «ИНФОРМАЦИОННОЕ ОБЕСПЕЧЕНИЕ ЭВМ И ИНФОРМАЦИОННЫЕ ТЕХНОЛОГИИ»

КУРСОВАЯ РАБОТА

НА ТЕМУ:

*Разработка базы данных для хранения наборов
данных (датасетов)*

Студент

ИУ7-62Б

(группа)

(подпись, дата)

(И.О. Фамилия)

Руководитель курсового
проекта

(подпись, дата)

Дроздецкая П.В.

(И.О. Фамилия)

Консультант

(подпись, дата)

(И.О. Фамилия)

2025 г.

Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
«Московский государственный технический университет имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

УТВЕРЖДАЮ

Заведующий кафедрой

ИУ7

(индекс)

И. В. Рудаков

(И.О. Фамилия)

28.02.2025 года

(дата)

(подпись)

З А Д А Н И Е на выполнение курсовой работы

по дисциплине «Базы данных»

Студент группы ИУ7-62Б Пантелеев Василий Максимович

(Фамилия, имя, отчество)

Тема курсовой работы Разработка базы данных для хранения наборов данных
(датасетов)

Направленность КР (учебная, исследовательская, практическая, др.) учебная

Источник тематики (кафедра,
предприятие, НИР)

кафедра

Задание

Провести анализ предметной области, определить основные сущности и их взаимосвязи. Сформулировать требования и ограничения к разрабатываемой базе данных. Спроектировать сущности базы данных и определить ограничения целостности данных. Спроектировать ролевую модель на уровне базы данных. Выбрать средства реализации базы данных и приложения. Разработать и реализовать сущности базы данных и ограничения целостности. Описать методы тестирования разработанного функционала и разработать тесты для проверки корректности работы приложения. Исследовать влияние применения различных типов индексов в базе данных на время выполнения запросов к базе данных.

Оформление курсовой работы:

Расчетно-пояснительная записка на 18-40 листах формата А4. Презентация к курсовой работе на 6-15 слайдах.

Дата выдачи задания «28» февраля 2025 г.

Руководитель курсовой работы

(подпись, дата)

Дроздецкая П.В.

(И.О. Фамилия)

Студент

(подпись, дата)

(И.О. Фамилия)

Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
«Московский государственный технический университет имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

КАЛЕНДАРНЫЙ ПЛАН
на выполнение курсовой работы

по дисциплине «Базы данных»

Студент группы ИУ7-62Б Пантелеев Василий Максимович
(Фамилия, имя, отчество)

Тема курсовой работы

Разработка базы данных для хранения наборов данных (датасетов)

№ п/п	Наименование этапов выпускной квалификационной работы	Сроки выполнения этапов		Отметка о выполнении	
		план	факт	Руководитель КП	Куратор
1.	Задание на выполнение курсовой работы			Дроздецкая П.В.	
2.	1 модуль	Планируемая дата		Дроздецкая П.В.	
3.	2 модуль	Планируемая дата		Дроздецкая П.В.	
4.	Оформление РПЗ (Отчета)	Планируемая дата		Дроздецкая П.В.	
5.	Подготовка доклада и презентации (при необходимости)	Планируемая дата		Дроздецкая П.В.	
6.	Защита курсовой работы	Планируемая дата		Дроздецкая П.В.	

Студент _____
(подпись, дата)

Руководитель работы _____
(подпись, дата)

РЕФЕРАТ

Курсовая работа 38 с., 8 рис., 15 табл., 17 ист., 1 прил.

БАЗА ДАННЫХ, ДАТАСЕТ, ВЕРСИОНИРОВАНИЕ, РОЛЕВАЯ МОДЕЛЬ, SQL, ИНДЕКСЫ.

Целью данной работы является разработка базы данных для хранения наборов данных (датасетов) и приложения для доступа к ней.

В рамках работы проведён анализ предметной области, выделены ключевые сущности и их взаимосвязи, сформулированы функциональные и нефункциональные требования. Спроектирована и реализована база данных с ограничениями целостности и ролевой моделью. Разработаны компоненты бизнес-логики, доступа к данным и пользовательский интерфейс. Проведено исследование влияния различных типов индексов на производительность выполнения запросов.

СОДЕРЖАНИЕ

РЕФЕРАТ	4
СОДЕРЖАНИЕ	5
ОБОЗНАЧЕНИЯ И СОКРАЩЕНИЯ	7
ВВЕДЕНИЕ	8
1 Аналитическая часть	9
1.1 Анализ предметной области	9
1.2 Анализ существующих решений	10
1.3 Анализ моделей данных	11
1.4 Формализация задачи	13
1.5 Формализация данных	14
1.6 Формализация ролей	15
2 Конструкторская часть	17
2.1 Формализация сущностей системы	17
2.2 Функция вычисления среднего рейтинга датасета	23
2.3 Описание триггера базы данных	24
2.4 Ролевая модель	25
3 Технологическая часть	28
3.1 Выбор системы управления данными	28
3.2 Средства реализации	29
3.3 Создание таблиц	29
3.4 Функция нахождения среднего рейтинга датасета	33
3.5 Реализация триггера базы данных	34
3.5.1 Тестирование триггера	36
3.6 Создание ролей	37

4	Исследовательская часть	40
4.1	Технические характеристики ЭВМ	40
4.2	Постановка задачи	40
4.3	Результаты замеров	42
	ЗАКЛЮЧЕНИЕ	47
	СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	48
	ПРИЛОЖЕНИЕ А	50

ОБОЗНАЧЕНИЯ И СОКРАЩЕНИЯ

В представленной расчетно-пояснительной записке используются следующие сокращения и обозначения:

- 1) БД – база данных;
- 2) СУБД – система управления базами данных;
- 3) ER – Entity Relationship;
- 4) API – Application Programming Interface;
- 5) SQL – Structured Query Language;
- 6) ЭВМ – электронно-вычислительная машина;
- 7) B-Tree – Binary tree.

ВВЕДЕНИЕ

В последние годы наблюдается стремительный рост количества датасетов, используемых в научных исследованиях, бизнес-аналитике и машинном обучении. Эти наборы данных часто хранятся в разрозненных источниках, без унифицированного подхода к работе с ними. Отсутствие централизованного и структурированного хранилища затрудняет повторное использование и совместную работу. Это порождает необходимость в разработке базы данных, ориентированной на хранение датасетов и управление ими.

Цель работы — разработка базы данных для хранения наборов данных (датасетов) и приложения для доступа к ней.

Для достижения поставленной цели требуется решить следующие задачи:

- провести анализ предметной области и определить ключевые сущности и их взаимосвязи;
- сформулировать функциональные и нефункциональные требования к системе;
- спроектировать структуру базы данных, включая ограничения целостности и ролевую модель;
- реализовать базу данных и необходимые механизмы доступа;
- провести исследование влияния различных типов индексов на производительность выполнения запросов.

1 Аналитическая часть

В данной части происходит анализ предметной области, обзор существующих аналогов, формализация хранимой информации, формулировка требований и ограничений, представлены ER-диаграмма сущностей, описание пользователей и диаграмма вариантов использования.

1.1 Анализ предметной области

В современном мире наборы данных (датасеты) стали основой для анализа, машинного обучения, научных исследований и бизнес-аналитики. Под датасетом понимается структурированная коллекция данных, используемая для анализа, обработки или обучения моделей машинного обучения. Датасет состоит из наблюдений (экземпляров) и признаков (характеристик), которые описывают каждое наблюдение. Такие наборы данных используются как входная информация для алгоритмов и моделей, а также для повторного анализа, тестирования и воспроизведения научных результатов [1].

На практике хранение и управление датасетами осуществляется с использованием как специализированных онлайн-платформ (например, Kaggle [2], Hugging Face [3], ClearML [4]), так и в локальных файловых системах. При этом часто возникают проблемы, связанные с отсутствием единых стандартов хранения, невозможностью эффективно отслеживать версии, а также слабо реализованной системой доступа по ролям. Это приводит к потере актуальности данных, дублированию и затруднениям в повторном использовании [5].

Для систем хранения датасетов важно учитывать следующие аспекты:

- версионирование — возможность сохранять историю изменений и откатываться к предыдущим версиям;
- метаданные — структурированная информация о датасете (описание, автор, дата создания, ключевые слова и пр.);
- связь с файлами — каждый датасет может состоять из одного или нескольких файлов разных форматов;
- права доступа — разграничение доступа для разных категорий пользователей (например, гостевой просмотр, авторское редактирование, администрирование);

— категоризация — удобная система разделения по категориям.

1.2 Анализ существующих решений

Для выявления ключевых требований к системе хранения и управления наборами данных, был проведён анализ четырёх популярных платформ: Kaggle, Hugging Face Datasets, UCI Machine Learning Repository и ClearML. Каждая из этих платформ имеет свою специфику, ориентирована на разные аудитории и задачи, и реализует различные подходы к организации данных.

- 1) **Kaggle Datasets** — платформа для проведения соревнований по анализу данных, а также для обмена наборами данных [2];
- 2) **Hugging Face Datasets** — открытая платформа для хранения, загрузки и использования наборов данных, преимущественно в области обработки естественного языка [3];
- 3) **UCI Machine Learning Repository** — один из старейших репозиторий академических наборов данных, ориентирован на предоставление исходных файлов для загрузки [6];
- 4) **ClearML** — платформа для управления процессами машинного обучения с возможностью отслеживания экспериментов и хранения наборов данных [4].

Критерии, выделенные для сравнения существующих решений:

- **Поддержка версий** — возможность хранения нескольких версий одного и того же датасета с отслеживанием изменений и восстановлением предыдущих состояний;
- **Отзывы и оценки** — наличие системы рейтингов, комментариев и обратной связи от пользователей, позволяющей оценивать качество и актуальность данных;
- **Уведомления об обновлениях** — рассылка автоматических оповещений о появлении новых версий датасетов подписанным пользователям;
- **Доступ к публичным датасетам** — наличие открытого доступа к общедоступным наборам данных без необходимости регистрации или подтверждения прав доступа.

Таблица 1.1 — Сравнение существующих платформ хранения датасетов

Платформа	Версионирование	Публичные датасеты	Отзывы и оценки	Уведомления об обновлениях
Kaggle	Нет	Да	Да	Нет
Hugging Face	Да	Да	Нет	Нет
UCI	Нет	Да	Нет	Нет
ClearML	Да	Нет	Нет	Да

Ни одно из вышеперечисленных существующих решений не удовлетворяет всем упомянутым критериям. Таким образом, отличием разрабатываемой системы является обеспечение всех выделенных критериев, что делает её универсальной и удобной для пользователей.

1.3 Анализ моделей данных

В данном разделе проводится обзор основных моделей данных, что необходимо для аргументированного выбора оптимального решения при разработке системы хранения и управления датасетами.

База данных – это самодокументированное собрание интегрированных записей. Под самодокументированностью базы данных понимается наличие в её составе метаданных — информации, описывающей структуру и организацию самих данных, включая таблицы, поля, типы данных и связи между сущностями. Интегрированность записей означает, что данные в базе логически связаны, согласованы и хранятся совместно с элементами, обеспечивающими их целостность и оптимальный доступ, такими как индексы и служебные структуры, описывающие взаимодействие с прикладными компонентами. [7]

Система управления базами данных (СУБД) — это набор программ, которые управляют структурой БД и контролируют доступ к данным, хранящимся в БД.

Логическая модель базы данных — версия структуры данных, на основе которой реализована конкретная СУБД.

По модели данных СУБД делятся на дореляционные, реляционные и постреляционные. [8]

Дореляционные модели

Дореляционные модели баз данных, к которым относятся иерархическая и сетевая, стали предшественниками реляционного подхода. В этих системах доступ к информации осуществлялся на уровне отдельных записей, и пользователи были вынуждены явно задавать маршруты навигации по структурам данных, как правило, через расширения языков программирования. [9]

Иерархическая модель представляла данные в виде древовидных структур с жёсткими связями «предок–потомок», что хорошо подходило для описания естественно иерархичных объектов. Сетевая модель, в свою очередь, расширила этот подход, позволив потомкам иметь несколько предков и тем самым лучше отражать сложные взаимосвязи между данными. Несмотря на появление реляционных систем, отдельные реализации дореляционных моделей сохраняют актуальность благодаря своей эффективности в специфических областях. [9]

Преимуществом дореляционных моделей является близость к естественным способам организации данных, что обеспечивало их эффективность в прикладных областях. Недостатком же являлась необходимость навигации на уровне отдельных записей и отсутствие развитых средств декларативного доступа.

Реляционные модели

Реляционные базы данных получили широкое распространение благодаря удобству представления информации в виде таблиц, состоящих из строк и столбцов, и применению строгого математического аппарата реляционной алгебры. Основные свойства реляционной модели включают уникальность строк, однородность данных в столбцах, наличие уникальных имён атрибутов и независимость порядка строк и столбцов. Такой подход обеспечивает простоту использования и гибкость в организации связей типа «один к одному» и «один ко многим».

Преимуществом реляционной модели является её универсальность и удобство декларативного доступа к данным, что позволило создать на её основе большинство современных СУБД. К ограничениям можно отнести снижение эффективности при работе с очень большими объёмами данных и необходимость сложной оптимизации запросов, что стимулировало развитие постреляционных

решений. [10]

Постреляционные модели

Постреляционная модель представляет собой дальнейшее развитие реляционного подхода и снимает ограничение на неделимость данных. В отличие от классической реляционной модели, она допускает использование многозначных полей, каждое из которых может содержать множество значений и рассматриваться как вложенная таблица. Это обеспечивает повышенную гибкость структуры данных, поскольку длина и количество полей в записях не фиксированы.

Основным преимуществом постреляционной модели является возможность объединения нескольких связанных реляционных таблиц в единую структуру, что делает представление информации более наглядным и повышает эффективность её обработки. Вместе с тем к недостаткам относятся сложности в обеспечении целостности и согласованности данных, что усложняет практическую реализацию таких систем. [9]

Выбор модели данных

Для реализации системы хранения датасетов целесообразно использовать комбинированный подход. Сами датасеты представляют собой крупные файлы различных форматов и хранятся во внешнем объектном хранилище, оптимизированном для работы с неструктурированными данными. Однако информация о них — сведения о версиях, категориях, тегах и пользователях, требуют строгой структуризации и поэтому размещаются в реляционной базе данных, обеспечивающей целостность и удобный декларативный доступ.

1.4 Формализация задачи

Необходимо спроектировать базу данных для хранения информации о датасетах, их версиях, связанных файлах, метаданных, тегах, категориях и пользователях. Требуется разработать приложение, предоставляющее интерфейс для просмотра, загрузки, редактирования и удаления информации, хранящейся в базе данных. Необходимо реализовать три вида ролей — гость, авторизованный пользователь и администратор.

В рамках поставленной цели необходимо спроектировать и реализовать

базу данных, содержащую информацию о датасетах и их структуре, поддерживающую версионирование, управление доступом и систему уведомлений об обновлениях. Приложение должно обеспечить возможность ведения метаданных и отслеживания изменений, а также предоставлять пользователю удобный интерфейс для взаимодействия с данными в рамках его роли.

1.5 Формализация данных

На основании ER-диаграммы были выделены следующие основные категории данных:

- Пользователи
- Подписки
- Категории
- Датасеты
- Версии датасетов
- Метаданные
- Отзывы

Таблица 1.2 — Формализация сущностей и их атрибутов

Категория	Сведения о данных (атрибуты)
Пользователь	Идентификатор, Имя пользователя, Пароль, Электронная почта, Страна, Дата регистрации
Категория	Идентификатор, Название, Описание
Датасет	Идентификатор, Название, Описание, Уровень доступа, Дата создания, Владелец, Категория
Подписка	Идентификатор, Дата создания, Датасет, Пользователь
Версия датасета	Идентификатор, Идентификатор датасета, Номер версии, Дата загрузки, Путь к файлу
Метаданные	Идентификатор, Идентификатор датасета, Формат, Размер, Теги
Отзыв	Идентификатор, Пользователь, Датасет, Текст отзыва, Оценка

На рисунке 1.1 приведена ER-диаграмма сущностей в нотации Чена.

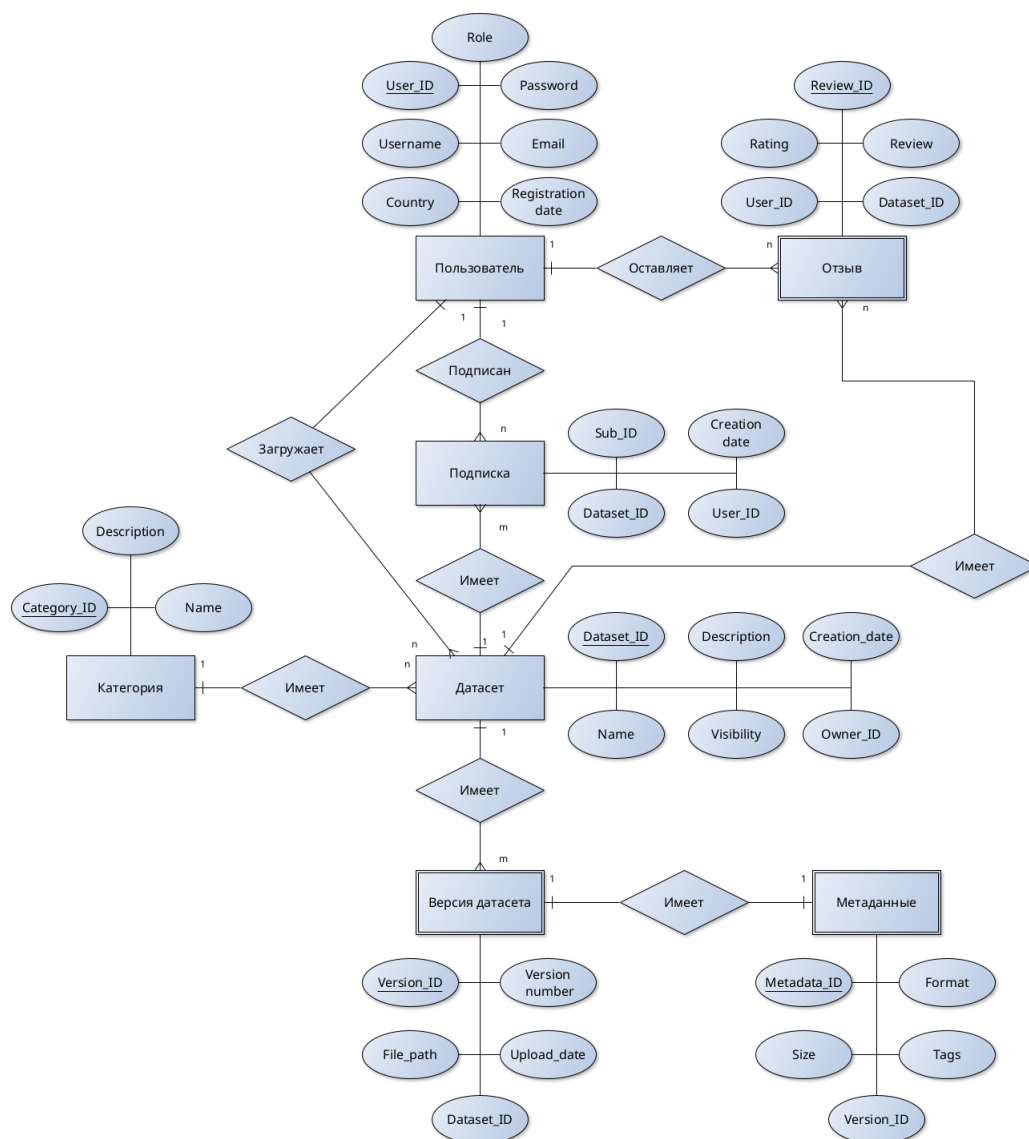


Рисунок 1.1 — ER-диаграмма сущностей в нотации Чена

1.6 Формализация ролей

Следует выделить группы пользователей разрабатываемой системы, исходя из предметной области поставленной задачи:

- гость — неавторизованный пользователь, который имеет возможность просматривать публичные датасеты и их метаданные а также скачивать их;
- авторизованный пользователь — пользователь, прошедший процедуру регистрации и авторизации. Может загружать собственные датасеты, редактировать их метаданные, создавать новые версии;
- администратор — обладает полными правами для управления всеми датасетами, метаданными, а также управляет правами доступа других пользователей, модерирует контент и контролирует работу системы.

На рисунке 1.2 приведена диаграмма вариантов использования.

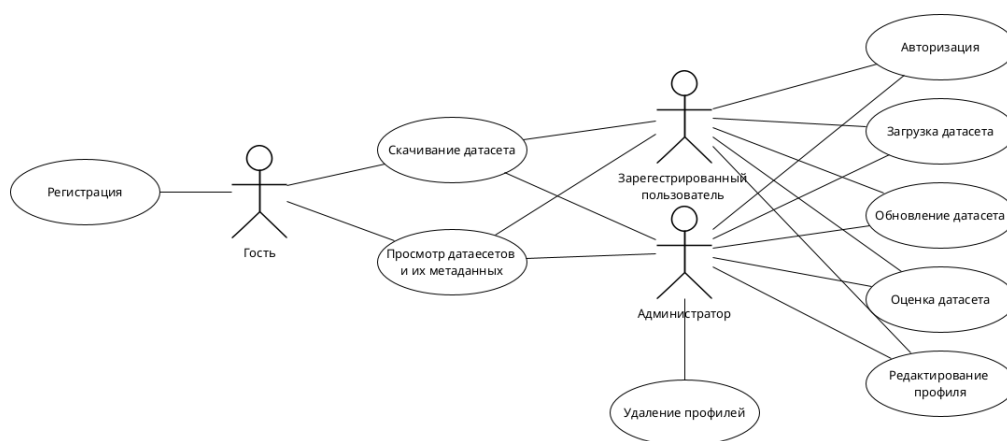


Рисунок 1.2 — Use-case диаграмма.

Вывод

В данном разделе была проанализирована предметная область, поставленная задача и рассмотрены существующие решения по хранению и управлению датасетами. Проведено сравнение популярных платформ, выявлены их достоинства и недостатки, а также сформулированы ключевые требования к разрабатываемой системе. Поскольку задача предполагает хранение взаимосвязанных структурированных данных, поддержку версионирования, отзывов, тегов и различных ролей пользователей, приоритетными свойствами модели данных являются целостность, расширяемость и независимость от пользовательского интерфейса.

2 Конструкторская часть

В данном разделе приводится проектирование базы данных: представлены её основные сущности и связи в виде ER-диаграммы, определены ограничения целостности, описана ролевая модель, а также приведена диаграмма классов для отражения структуры и взаимодействия элементов системы.

2.1 Формализация сущностей системы

На основе анализа предметной области, проведённого в предыдущем разделе, выделены ключевые сущности разрабатываемой базы данных. Каждая сущность представлена в виде таблицы, содержащей атрибуты с указанными типами данных, ограничениями целостности и связями с другими таблицами.

В реализуемой базе данных выделены следующие таблицы:

- **Users** – таблица пользователей системы;
- **Categories** – таблица категорий датасетов;
- **Datasets** – таблица с основной информацией о датасетах;
- **Dataset_Versions** – таблица версий датасетов;
- **Metadata** – таблица метаданных, связанных с версиями датасетов;
- **Subscriptions** – таблица подписок пользователей на датасеты;
- **Reviews** – таблица отзывов и оценок пользователей;
- **Notifications** – таблица уведомлений пользователей.

На рисунке 2.1 представлена диаграмма сущность-связь базы данных

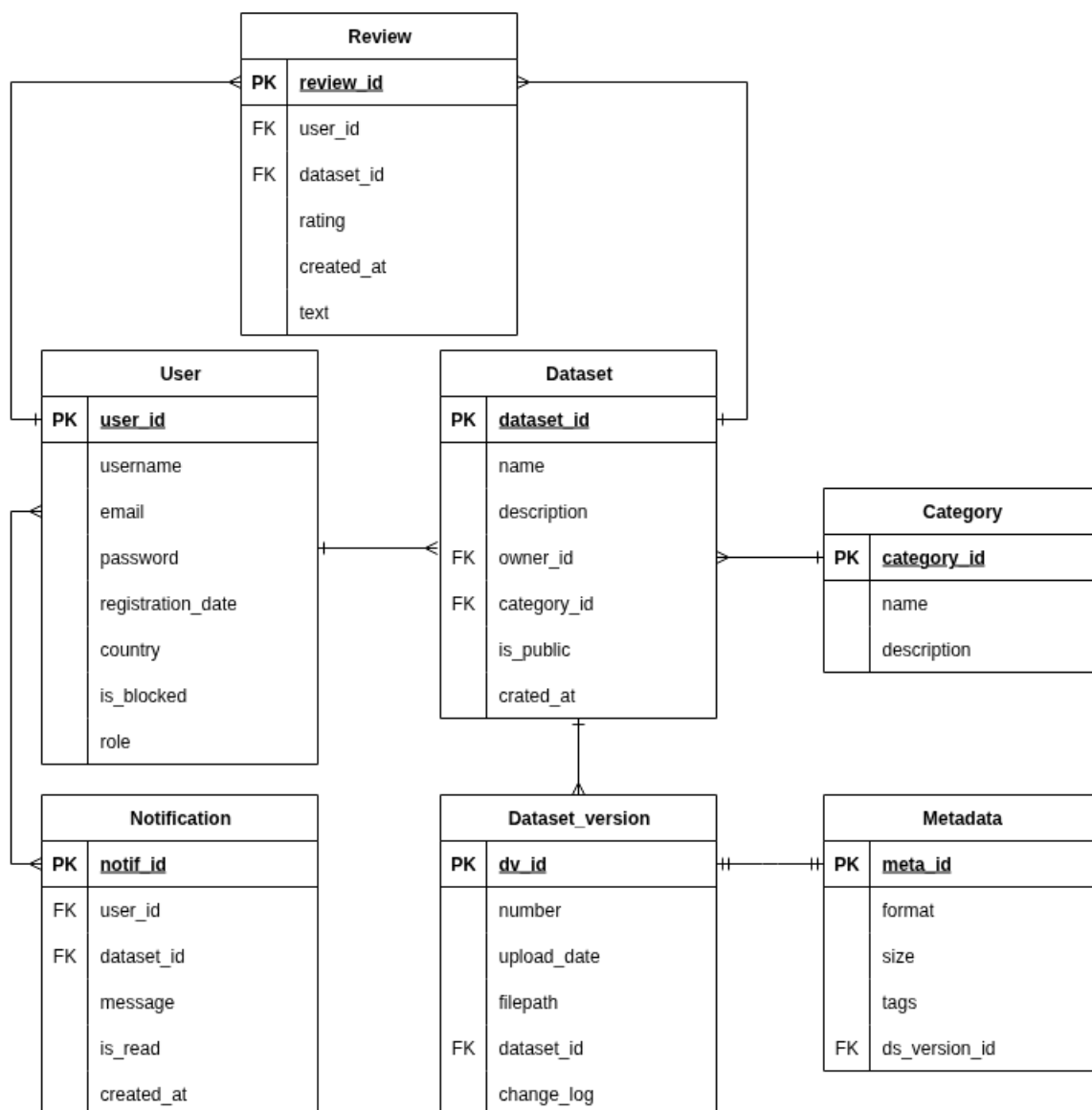


Рисунок 2.1 — ER-диаграмма базы данных

В таблицах 2.1 – 2.8 приведены поля таблиц базы данных.

Таблица Users

Таблица 2.1 — Атрибуты таблицы Users

Атрибут	Тип	Ограничение	Описание
id	целое число	первичный ключ	Уникальный идентификатор пользователя
username	строка	обязательное поле	Имя пользователя
email	строка	обязательное поле, уникальное значение	Электронная почта пользователя
password	строка	обязательное поле	Хеш пароля пользователя
registration_date	дата/время	обязательное поле, значение по умолчанию: текущие дата и время	Дата и время регистрации
country	строка	обязательное поле	Страна пользователя
is_blocked	логический	обязательное поле, значение по умолчанию: false	Статус блокировки пользователя
role	строка	обязательное поле, допустимые значения: <i>guest</i> , <i>user</i> , <i>admin</i>	Роль пользователя в системе

Таблица Categories

Таблица 2.2 — Атрибуты таблицы Categories

Атрибут	Тип	Ограничение	Описание
id	целое число	первичный ключ	Уникальный идентификатор категории
name	строка	обязательное поле, уникальное значение	Название категории
description	текст	поле может отсутствовать	Описание категории

Таблица Datasets

Таблица 2.3 — Атрибуты таблицы Datasets

Атрибут	Тип	Ограничение	Описание
id	целое число	первичный ключ	Уникальный идентификатор датасета
name	строка	обязательное поле	Название датасета
description	текст	поле может отсутствовать	Краткое описание датасета
owner_id	целое число	внешний ключ	Владелец датасета
category_id	целое число	внешний ключ	Категория датасета
is_public	логический	обязательное поле, значение по умолчанию: false	Признак публичного доступа
created_at	дата/время	обязательное поле, значение по умолчанию: текущие дата и время	Дата создания

Таблица Dataset_Versions

Таблица 2.4 — Атрибуты таблицы Dataset_Versions

Атрибут	Тип	Ограничение	Описание
id	целое число	первичный ключ	Уникальный идентификатор версии
number	строка	обязательное поле	Номер версии
upload_date	дата/время	обязательное поле, значение по умолчанию: текущие дата и время	Дата загрузки версии
filepath	текст	обязательное поле	Путь к файлу версии
dataset_id	целое число	внешний ключ	Датасет, к которому относится версия
change_log	текст	поле может отсутствовать	Описание изменений в версии

Таблица Metadata

Таблица 2.5 — Атрибуты таблицы Metadata

Атрибут	Тип	Ограничение	Описание
id	целое число	первичный ключ	Уникальный идентификатор метаданных
format	строка	обязательное поле	Формат данных (например, CSV, JSON и др.)
size	целое число	обязательное поле, значение должно быть положительным	Размер в байтах
tags	текст	поле может отсутствовать	Список ключевых слов (тегов)
dataset version id	целое число	внешний ключ	Версия датасета, к которой относятся метаданные

Таблица Subscriptions

Таблица 2.6 — Атрибуты таблицы Subscriptions

Атрибут	Тип	Ограничение	Описание
user_id	целое число	внешний ключ, составной первичный ключ	Пользователь, оформивший подписку
dataset_id	целое число	внешний ключ, составной первичный ключ	Датасет, на который оформлена подписка
created_at	дата/время	обязательное поле, значение по умолчанию: текущие дата и время	Дата создания подписки

Таблица Reviews

Таблица 2.7 — Атрибуты таблицы Reviews

Атрибут	Тип	Ограничение	Описание
id	целое число	первичный ключ	Уникальный идентификатор отзыва
user_id	целое число	внешний ключ	Пользователь, оставивший отзыв
dataset_id	целое число	внешний ключ	Датасет, к которому относится отзыв
rating	целое число	обязательное поле, допустимый диапазон: от 1 до 5	Оценка качества датасета
created_at	дата/время	обязательное поле, значение по умолчанию: текущие дата и время	Дата создания отзыва
text	текст	поле может отсутствовать	Текстовый комментарий

Таблица Notifications

Таблица 2.8 — Атрибуты таблицы Notifications

Атрибут	Тип	Ограничение	Описание
id	целое число	первичный ключ	Уникальный идентификатор уведомления
user_id	целое число	внешний ключ	Пользователь, которому адресовано уведомление
dataset_id	целое число	внешний ключ	Датасет, связанный с уведомлением
message	текст	обязательное поле	Текст уведомления
is_read	логический	обязательное поле, значение по умолчанию: false	Признак прочтения уведомления
created_at	дата/время	обязательное поле, значение по умолчанию: текущие дата и время	Дата создания уведомления

2.2 Функция вычисления среднего рейтинга датасета

В рамках базы данных реализована функция вычисления среднего значения рейтингов для конкретного датасета. На вход функция принимает идентификатор датасета. Если оценки присутствуют, функция вычисляет их среднее арифметическое в диапазоне от 1 до 5. В случае отсутствия оценок результатом работы функции является 0.0.

На рисунке 2.2 представлено описание алгоритма функции вычисления среднего рейтинга датасета.

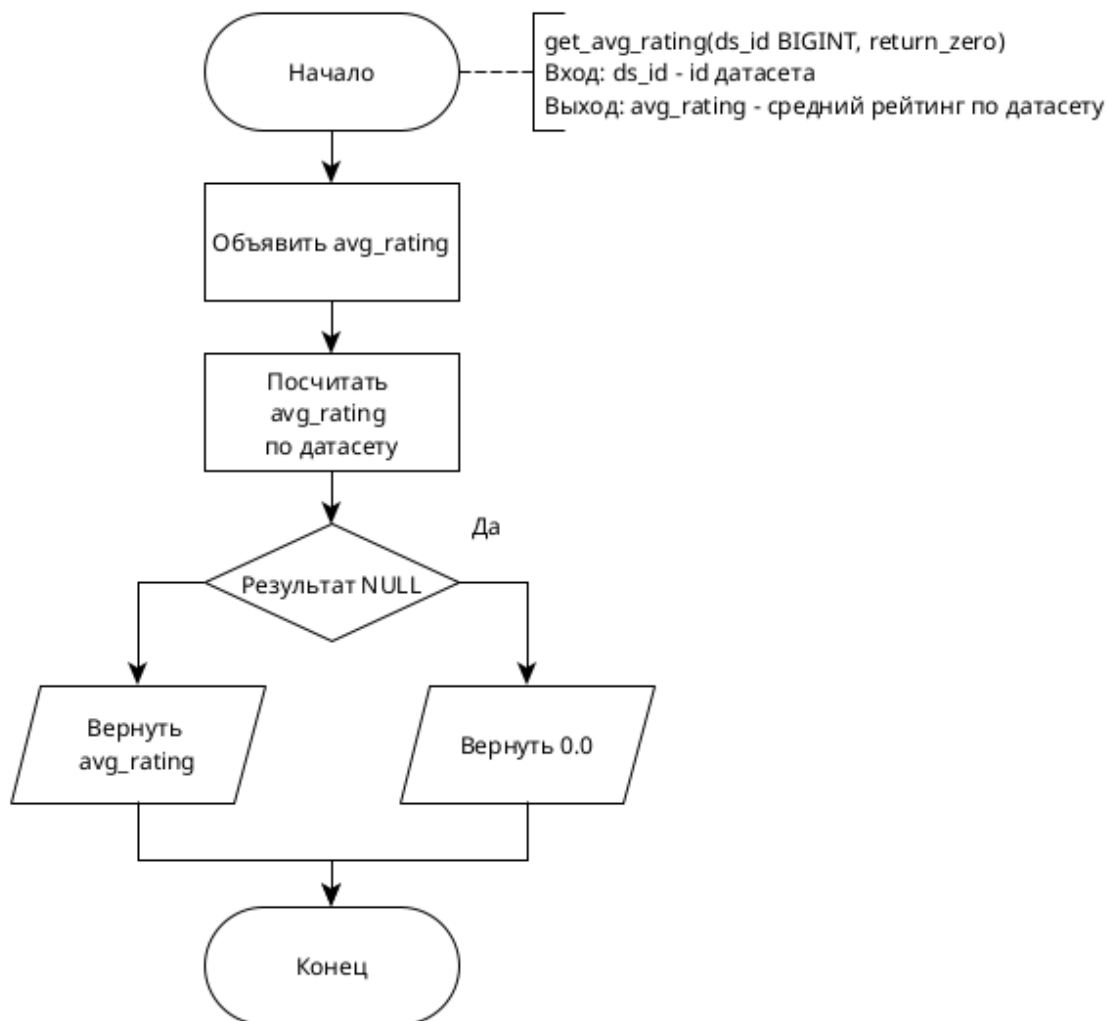


Рисунок 2.2 — Схема алгоритма функции вычисления среднего рейтинга

2.3 Описание триггера базы данных

В базе данных реализован триггер, который срабатывает после добавления новой записи в таблицу datasets. Его назначение — автоматизация служебных действий при создании датасета: проверка владельца, настройка видимости, создание подписки, инициализация версии и уведомление владельца.

На рисунке 2.3 представлено описание алгоритма триггера.

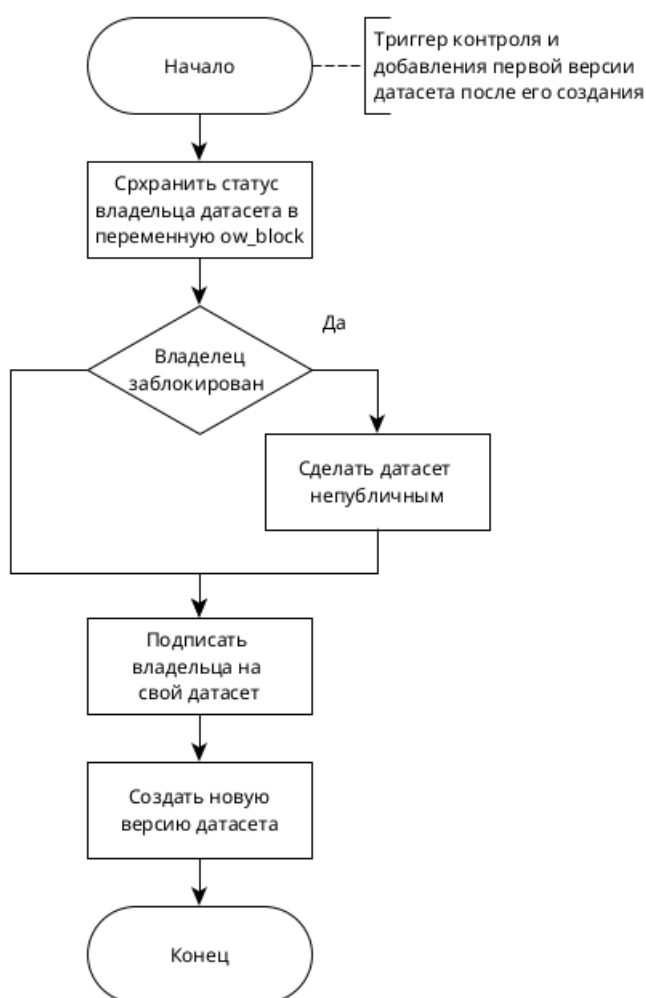


Рисунок 2.3 — Схема алгоритма работы триггера базы данных

2.4 Ролевая модель

Для обеспечения безопасности и управления доступом к данным в системе разработана ролевая модель. Она определяет права доступа для различных категорий пользователей, что позволяет ограничить доступ к данным в зависимости от их роли, минимизировать риски несанкционированного изменения информации и упростить администрирование системы.

Роль Guest (Гость)

Гость — это неавторизованный пользователь, который может только просматривать общедоступную информацию. Права:

— SELECT — над таблицей datasets (просмотр только записей с атрибу-

том `is_public = TRUE`);

— `SELECT` — над таблицей `categories` (просмотр категорий).

Гость не имеет доступа к данным о пользователях, подписках, отзывах, уведомлениях и версиях датасетов. Это обеспечивает защиту персональных данных и ограничивает функционал незарегистрированных пользователей.

Роль User (Зарегистрированный пользователь)

Зарегистрированный пользователь имеет более широкий набор прав: он может подписываться на датасеты, оставлять отзывы и управлять собственными наборами данных. Права:

— `SELECT` — над таблицами `datasets`, `categories`, `dataset_versions`, `metadata` (просмотр информации о датасетах);

— `SELECT`, `INSERT` — над таблицей `subscriptions` (оформление подписки и просмотр своих подписок);

— `SELECT`, `INSERT` — над таблицей `reviews` (просмотр и добавление отзывов);

— `SELECT`, `INSERT` — над таблицей `notifications` (получение и отметка уведомлений);

— `SELECT`, `INSERT`, `UPDATE`, `DELETE` — над таблицей `datasets`, но только для записей, где он является владельцем (`owner_id = current_user`);

— `SELECT`, `INSERT`, `UPDATE`, `DELETE` — над таблицей `dataset_versions` для версий собственных датасетов;

— `SELECT`, `INSERT`, `UPDATE`, `DELETE` — над таблицей `metadata`, связанной с собственными версиями датасетов.

Таким образом, пользователь имеет полный контроль только над своими данными, а над чужими датасетами и их версиями — только права чтения, если они публичны.

Роль Admin (Администратор)

Администратор — это пользователь с полным доступом ко всем объектам системы. Права:

— `SELECT`, `INSERT`, `UPDATE`, `DELETE` — над таблицей `users` (управле-

ние учётными записями);

— SELECT, INSERT, UPDATE, DELETE — над таблицами datasets, dataset_versions, metadata (полный контроль над хранимыми наборами данных);

— SELECT, INSERT, UPDATE, DELETE — над таблицами categories, subscriptions, reviews, notifications;

— Возможность блокировки пользователей (is_blocked = TRUE);

— Возможность изменения ролей пользователей (role в таблице users).

Администратор отвечает за модерацию содержимого, управление пользователями, категориями и обеспечение корректного функционирования системы в целом.

Вывод

В конструкторской части была спроектирована структура базы данных: выделены ключевые сущности и их атрибуты, определены ограничения целостности и взаимосвязи между таблицами. Построена ER-диаграмма, наглядно отражающая архитектуру системы. Для обеспечения безопасности и разграничения прав доступа разработана ролевая модель, включающая роли гостя, зарегистрированного пользователя и администратора. Описанные элементы создают основу для дальнейшей реализации и тестирования базы данных и прикладного программного обеспечения.

3 Технологическая часть

В данном разделе рассматривается процесс выбора инструментов для реализации базы данных и прикладного программного обеспечения. Обосновывается выбор системы управления базами данных и используемого технологического стека. Описывается создание сущностей базы данных, заданных ограничений целостности и пользовательских ролей. Приводятся SQL-скрипты для создания таблиц и ролей в разработанной системе, а также тестируется триггер.

3.1 Выбор системы управления данными

Для реализации системы хранения и каталогизации датасетов требуется база данных для метаданных и отдельное хранилище для самих файлов. Для сравнения были выбраны несколько популярных СУБД.

Сравнение выполнено по четырём простым критериям:

- поддержка ролевой модели;
- поддержка ограничений целостности;
- поддержка пользовательских типов данных.
- наличие средств резервного копирования и восстановления.

Таблица 3.1 — Сравнение СУБД по расширенным критериям

СУБД	Роли	Ограничения	Индексы	Бэкап	Польз. типы
PostgreSQL	есть	есть	есть	есть	
MySQL	есть	есть	есть	ограниченно	
SQLite	нет	частично	файл	нет	

SQLite [11] — простая встраиваемая СУБД, которая не поддерживает полноценную ролевую модель и имеет ограниченные механизмы целостности. Подходит для локальных приложений, но не для многопользовательской системы.

MySQL [12] является одной из наиболее распространённых СУБД и поддерживает ролевую модель, индексы и базовые ограничения целостности. Однако её возможности по расширяемости и созданию пользовательских типов данных значительно уступают PostgreSQL.

Таким образом, для метаданных выбран **PostgreSQL** [13] как наиболее сбалансированное решение.

Для хранения самих датасетов целесообразно использовать **объектное хранилище S3** [14], так как оно рассчитано на работу с большими бинарными файлами, обеспечивает высокую надёжность хранения и простую интеграцию через стандартный протокол.

3.2 Средства реализации

Для реализации программного обеспечения системы хранения датасетов был выбран язык программирования **Go** [15]. Данный выбор обусловлен его строгой статической типизацией и поддержкой всех необходимых типов данных, определённых на этапе проектирования базы данных. Язык также обеспечивает удобные механизмы работы с многопоточностью и сетевыми операциями, что делает его подходящим для создания серверных приложений.

Для взаимодействия между серверной частью и пользователями реализован API на базе стандартной библиотеки Go и фреймворка net/http, что предоставляет возможность интеграции с внешними системами и клиентскими приложениями.

3.3 Создание таблиц

В данном подразделе приведены SQL-скрипты создания таблиц базы данных, а также последующего добавления ограничений целостности.

Таблица Users

Листинг 3.1 — Создание таблицы Users

```
1 CREATE TABLE users (  
2     id                BIGSERIAL PRIMARY KEY,  
3     username          VARCHAR(50)  NOT NULL,  
4     email              VARCHAR(255) NOT NULL,  
5     password           VARCHAR(255) NOT NULL,  
6     registration_date TIMESTAMPTZ  NOT NULL DEFAULT now(),  
7     country            VARCHAR(100) NOT NULL,  
8     is_blocked         BOOLEAN      NOT NULL DEFAULT FALSE,  
9     role               VARCHAR(10)  NOT NULL  
10 );  
11
```

```

12 ALTER TABLE users
13     ADD CONSTRAINT uq_users_email UNIQUE (email),
14     ADD CONSTRAINT chk_users_role CHECK (role IN
        ('guest', 'user', 'admin'));

```

Таблица Categories

Листинг 3.2 — Создание таблицы Categories

```

1 CREATE TABLE categories (
2     id          BIGSERIAL PRIMARY KEY,
3     name        VARCHAR(50) NOT NULL,
4     description TEXT
5 );
6
7 ALTER TABLE categories
8     ADD CONSTRAINT uq_categories_name UNIQUE (name);

```

Таблица Datasets

Листинг 3.3 — Создание таблицы Datasets

```

1 CREATE TABLE datasets (
2     id          BIGSERIAL PRIMARY KEY,
3     name        VARCHAR(100) NOT NULL,
4     description TEXT,
5     owner_id    BIGINT NOT NULL,
6     category_id BIGINT NOT NULL,
7     is_public   BOOLEAN      NOT NULL DEFAULT FALSE,
8     created_at  TIMESTAMPTZ  NOT NULL DEFAULT now()
9 );
10
11 ALTER TABLE datasets
12     ADD CONSTRAINT fk_datasets_owner FOREIGN KEY
13     (owner_id) REFERENCES users(id) ON DELETE RESTRICT,
14     ADD CONSTRAINT fk_datasets_category FOREIGN KEY
15     (category_id) REFERENCES categories(id) ON DELETE RESTRICT;
16
17 CREATE INDEX idx_datasets_owner ON datasets(owner_id);
18 CREATE INDEX idx_datasets_category_public_created

```

```
19 ON datasets(category_id, is_public, created_at DESC);
```

Таблица Dataset_Versions

Листинг 3.4 — Создание таблицы Dataset_Versions

```
1 CREATE TABLE dataset_versions (  
2     id                BIGSERIAL PRIMARY KEY,  
3     number            VARCHAR(20) NOT NULL,  
4     upload_date       TIMESTAMPTZ NOT NULL DEFAULT now(),  
5     filepath          TEXT          NOT NULL,  
6     dataset_id        BIGINT        NOT NULL,  
7     change_log        TEXT  
8 );  
9  
10 ALTER TABLE dataset_versions  
11     ADD CONSTRAINT fk_versions_dataset FOREIGN KEY  
12     (dataset_id) REFERENCES datasets(id) ON DELETE CASCADE;  
13  
14 CREATE INDEX idx_versions_dataset ON dataset_versions(dataset_id,  
    upload_date DESC);
```

Таблица Metadata

Листинг 3.5 — Создание таблицы Metadata

```
1 CREATE TABLE metadata (  
2     id                BIGSERIAL PRIMARY KEY,  
3     format            VARCHAR(50) NOT NULL,  
4     size              BIGINT        NOT NULL,  
5     tags              TEXT,  
6     dataset_version_id BIGINT        NOT NULL  
7 );  
8  
9 ALTER TABLE metadata  
10     ADD CONSTRAINT chk_metadata_size CHECK (size > 0),  
11     ADD CONSTRAINT fk_metadata_version FOREIGN KEY  
12     (dataset_version_id) REFERENCES dataset_versions(id) ON  
    DELETE CASCADE;  
13
```

```
14 CREATE INDEX idx_metadata_version ON metadata(dataset_version_id);
```

Таблица Subscriptions

Листинг 3.6 — Создание таблицы Subscriptions

```
1 CREATE TABLE subscriptions (  
2     user_id      BIGINT NOT NULL,  
3     dataset_id   BIGINT NOT NULL,  
4     created_at   TIMESTAMPTZ NOT NULL DEFAULT now() ,  
5     PRIMARY KEY (user_id , dataset_id)  
6 );  
7  
8 ALTER TABLE subscriptions  
9     ADD CONSTRAINT fk_subscriptions_user FOREIGN KEY  
10    (user_id) REFERENCES users(id) ON DELETE CASCADE,  
11    ADD CONSTRAINT fk_subscriptions_dataset FOREIGN KEY  
12    (dataset_id) REFERENCES datasets(id) ON DELETE CASCADE;  
13  
14 CREATE INDEX idx_subscriptions_dataset ON  
    subscriptions(dataset_id);
```

Таблица Reviews

Листинг 3.7 — Создание таблицы Reviews

```
1 CREATE TABLE reviews (  
2     id           BIGSERIAL PRIMARY KEY,  
3     user_id      BIGINT  NOT NULL,  
4     dataset_id   BIGINT  NOT NULL,  
5     rating       SMALLINT NOT NULL,  
6     created_at   TIMESTAMPTZ NOT NULL DEFAULT now() ,  
7     text         TEXT  
8 );  
9  
10 ALTER TABLE reviews  
11     ADD CONSTRAINT fk_reviews_user FOREIGN KEY  
12     (user_id) REFERENCES users(id) ON DELETE CASCADE,  
13     ADD CONSTRAINT fk_reviews_dataset FOREIGN KEY  
14     (dataset_id) REFERENCES datasets(id) ON DELETE CASCADE,
```



```

15      ADD CONSTRAINT chk_reviews_rating CHECK (rating BETWEEN 1 AND
        5);
16
17 CREATE INDEX idx_reviews_dataset ON reviews(dataset_id);
18 CREATE INDEX idx_reviews_user ON reviews(user_id);

```

Таблица Notifications

Листинг 3.8 — Создание таблицы Notifications

```

1 CREATE TABLE notifications (
2     id          BIGSERIAL PRIMARY KEY,
3     user_id     BIGINT NOT NULL,
4     dataset_id  BIGINT NOT NULL,
5     message     TEXT    NOT NULL,
6     is_read     BOOLEAN NOT NULL DEFAULT FALSE,
7     created_at  TIMESTAMPTZ NOT NULL DEFAULT now()
8 );
9
10 ALTER TABLE notifications
11     ADD CONSTRAINT fk_notifications_user FOREIGN KEY
12     (user_id) REFERENCES users(id) ON DELETE CASCADE,
13     ADD CONSTRAINT fk_notifications_dataset FOREIGN KEY
14     (dataset_id) REFERENCES datasets(id) ON DELETE CASCADE;
15
16 CREATE INDEX idx_notifications_user ON notifications(user_id);

```

3.4 Функция нахождения среднего рейтинга датасета

На листинге 3.9 представлена реализация функции базы данных для нахождения среднего рейтинга датасета.

Листинг 3.9 — Реализация функции базы данных

```

1 CREATE
2 OR REPLACE FUNCTION get_dataset_avg_rating(
3     p_dataset_id BIGINT,
4     p_return_zero_if_empty BOOLEAN DEFAULT TRUE
5 )
6 RETURNS NUMERIC(10,4)
7 LANGUAGE plpgsql

```

```

8 AS $$
9 DECLARE
10 avg_rating NUMERIC(10,4);
11 BEGIN
12 SELECT AVG(rating) ::NUMERIC(10,4)
13 INTO avg_rating
14 FROM ratings
15 WHERE dataset_id = p_dataset_id
16    AND rating BETWEEN 1 AND 5;
17
18 IF
19 avg_rating IS NULL THEN
20     IF p_return_zero_if_empty THEN
21         RETURN 0.0;
22     ELSE
23         RETURN NULL;
24 END IF;
25 END IF;
26
27 RETURN avg_rating;
28 END;
29 $$;

```

3.5 Реализация триггера базы данных

На листинге 3.10 представлена реализация триггера базы данных для автоматизации служебных действий при создании датасета.

Листинг 3.10 — Реализация триггера базы данных

```

1 CREATE OR REPLACE FUNCTION trg_after_insert_datasets()
2 RETURNS TRIGGER
3 LANGUAGE plpgsql
4 AS $$
5 DECLARE
6 v_owner_role    VARCHAR(10);
7   v_owner_block  BOOLEAN;
8   v_now          TIMESTAMPTZ := now();
9   v_forced_msg   TEXT := NULL;
10   v_placeholder  TEXT := format('pending://upload/%s', NEW.id);
11 BEGIN

```

```

12
13 SELECT role , is_blocked
14 INTO v_owner_role , v_owner_block
15 FROM users
16 WHERE id = NEW.owner_id;
17
18 IF v_owner_role IS NULL THEN
19     RAISE EXCEPTION 'Owner % not found for dataset %',
20         NEW.owner_id , NEW.id;
21
22     IF v_owner_block IS TRUE AND NEW.is_public IS TRUE THEN
23 UPDATE datasets SET is_public = FALSE WHERE id = NEW.id;
24 v_forced_msg := 'Dataset visibility forced to private: owner is
25     blocked';
26
27 END IF;
28
29 INSERT INTO subscriptions(user_id , dataset_id , created_at)
30 VALUES (NEW.owner_id , NEW.id , v_now)
31 ON CONFLICT (user_id , dataset_id) DO NOTHING;
32
33 IF NOT EXISTS (SELECT 1 FROM dataset_versions WHERE dataset_id =
34     NEW.id) THEN
35     INSERT INTO dataset_versions(number , upload_date ,
36         filepath , dataset_id , change_log)
37     VALUES ('v0.1' , v_now , v_placeholder , NEW.id ,
38         'Initial version placeholder (created by
39         trigger)');
40
41 END IF;
42
43 INSERT INTO notifications(user_id , dataset_id , message , is_read ,
44     created_at)
45 VALUES (
46     NEW.owner_id ,
47     NEW.id ,
48     COALESCE(
49         v_forced_msg ,
50         'Dataset created. Upload the file to replace
51         placeholder URL'
52     ) ,

```

```

45         FALSE ,
46         v_now
47     );
48
49 RETURN NEW;
50 END;
51 $$;
52
53 DROP TRIGGER IF EXISTS trg_after_insert_datasets ON datasets;
54
55 CREATE TRIGGER trg_after_insert_datasets
56     AFTER INSERT ON datasets
57     FOR EACH ROW
58     EXECUTE FUNCTION trg_after_insert_datasets();

```

3.5.1 Тестирование триггера

Для тестирования работы триггера были выделены классы эквивалентности, отражающие все возможные ситуации при вставке датасета.

Классы эквивалентности

Таблица 3.2 — Классы эквивалентности для триггера `trg_after_insert_datasets`

Условие	Класс эквивалентности	Ожидаемый результат
Владелец существует, не заблокирован	Нормальный ввод	Создаются подписка владельца на датасет и первая версия датасета.
Владелец существует, но заблокирован, датасет создан публичным	Владелец заблокирован + публичный датасет	Поле <code>is_public</code> принудительно устанавливается в <code>FALSE</code> , создаются подписка и версия.
Владелец существует, но заблокирован, датасет создан приватным	Владелец заблокирован + приватный датасет	Датасет остаётся приватным, создаются подписка и версия.
Владелец не найден в таблице <code>users</code>	Невалидный ввод (несуществующий владелец)	Выбрасывается исключение: <code>Owner <id> not found for dataset <id>.</code>

3.6 Создание ролей

Для обеспечения безопасности и разграничения доступа к данным в базе данных разработана ролевая модель. В неё входят три роли: *гость*, *зарегистрированный пользователь* и *администратор*. Ниже приведены SQL-скрипты, реализующие данную модель.

Роль Guest

Листинг 3.11 — Создание роли Guest

```
1 CREATE ROLE guest_role WITH
2     NOSUPERUSER
3     NOCREATEDB
4     NOCREATEROLE
5     NOINHERIT
6     NOREPLICATION
7     NOBYPASSRLS
8     CONNECTION LIMIT -1
9     LOGIN
10    PASSWORD 'Guestpass';
11
12 GRANT SELECT ON datasets TO guest_role;
13
14 ALTER TABLE datasets ENABLE ROW LEVEL SECURITY;
15
16 CREATE POLICY guest_datasets_select ON datasets
17     FOR SELECT TO guest_role
18     USING (true);
```

Роль Registered User

Листинг 3.12 — Создание роли Registered User

```
1 CREATE ROLE registered_role WITH
2     NOSUPERUSER
3     NOCREATEDB
4     NOCREATEROLE
5     NOINHERIT
6     NOREPLICATION
```

```

7      NOBYPASSRLS
8      CONNECTION LIMIT -1
9      LOGIN
10     PASSWORD 'Userpass';
11
12 GRANT SELECT ON datasets , versions , users TO registered_role;
13 GRANT INSERT ON datasets TO registered_role;
14 GRANT UPDATE, DELETE ON datasets TO registered_role;
15
16 ALTER TABLE datasets ENABLE ROW LEVEL SECURITY;
17 ALTER TABLE files ENABLE ROW LEVEL SECURITY;
18
19 CREATE POLICY user_datasets_policy ON datasets
20     FOR ALL TO registered_role
21     USING (owner_id = current_user::TEXT);

```

Роль Admin

Листинг 3.13 — Создание роли Admin

```

1 CREATE ROLE admin_role WITH
2     NOSUPERUSER
3     NOCREATEDB
4     NOCREATEROLE
5     INHERIT
6     NOREPLICATION
7     NOBYPASSRLS
8     CONNECTION LIMIT -1
9     LOGIN
10    PASSWORD 'Adminpass';
11
12 GRANT ALL PRIVILEGES ON ALL TABLES IN SCHEMA public TO admin_role;
13 GRANT ALL PRIVILEGES ON ALL SEQUENCES IN SCHEMA public TO
14     admin_role;
15 GRANT ALL PRIVILEGES ON ALL FUNCTIONS IN SCHEMA public TO
16     admin_role;
17 GRANT ALL PRIVILEGES ON ALL PROCEDURES IN SCHEMA public TO
18     admin_role;
19
20 ALTER ROLE admin_role BYPASSRLS;

```

Таким образом, ролевая модель обеспечивает:

- доступ на чтение для гостей;
- ограниченный доступ с возможностью модификации только собственных данных для зарегистрированных пользователей;
- полный контроль над всеми объектами базы данных для администраторов.

Вывод

В технологической части был выполнен выбор средств реализации программного обеспечения и базы данных, реализованы основные сущности и ограничения целостности, а также разработана ролевая модель. Приведены листинги функции и триггера базы данных, проведено их тестирование с использованием классов эквивалентности.

4 Исследовательская часть

В данном разделе исследуется влияние разных типов индексирования на время выполнения базовых операций с таблицей: вставки, выборки и удаления. Цель — показать, как выбор индекса влияет на производительность и скорость выполнения основных операций

4.1 Технические характеристики ЭВМ

Все замеры проводились на ЭВМ, характеристики которой приведены ниже:

- процессор: AMD Ryzen™ 7–5800H with Radeon™ Graphics × 16, 3.8 ГГц [16];
- ОС: 64-разрядная операционная система Ubuntu 24.04.1 LTS [17];
- оперативная память: 16,0 ГБ;
- число логических ядер – 16;
- СУБД: PostgreSQL.

4.2 Постановка задачи

Цель исследования — измерить зависимость времени выполнения INSERT, SELECT и DELETE от используемого типа индексирования над столбцом k в одной и той же таблице. Рассматриваемые сценарии:

1. **Без индекса (Heap)** — таблица-куча без индексов (см. реализацию 4.1).
2. **B-Tree** — некластерный индекс по атрибуту k (см. реализацию 4.2).
3. **UNIQUE B-Tree** — уникальный индекс по атрибуту k (см. реализацию 4.3).
4. **CLUSTER по k** — кластеризованная (физически упорядоченная) укладка таблицы на основе B-Tree (см. реализацию 4.4).

Листинг 4.1 — Сценарий без индекса

```
1 type noIndex struct{ pool *pgxpool.Pool }
2
3 func (s noIndex) Name() string { return "heap_no_index" }
4 func (s noIndex) Prepare(ctx context.Context) {
5     mustExec(ctx, s.pool, 'DROP INDEX IF EXISTS idx_bench_k;')
6 }
```



```

7 func (s noIndex) Reset(ctx context.Context) {
8     mustExec(ctx, s.pool, 'DROP INDEX IF EXISTS idx_bench_k;')
9 }
10 func (s noIndex) PostInsertLayout(ctx context.Context) {
11 }
12 func (s noIndex) Cleanup(ctx context.Context) {
13     mustExec(ctx, s.pool, 'DROP INDEX IF EXISTS idx_bench_k;')
14 }

```

Листинг 4.2 — Сценарий B-Tree по атрибуту k

```

1 type heapBtree struct{ pool *pgxpool.Pool }
2
3 func (s heapBtree) Name() string { return "heap_btree" }
4 func (s heapBtree) Prepare(ctx context.Context) {
5     mustExec(ctx, s.pool, 'DROP INDEX IF EXISTS idx_bench_k;')
6 }
7 func (s heapBtree) Reset(ctx context.Context) {
8     mustExec(ctx, s.pool, 'DROP INDEX IF EXISTS idx_bench_k;')
9     mustExec(ctx, s.pool, 'CREATE INDEX idx_bench_k ON
    bench_items(k);')
10 }
11 func (s heapBtree) PostInsertLayout(ctx context.Context) {
12     mustExec(ctx, s.pool, 'ANALYZE bench_items;')
13 }
14 func (s heapBtree) Cleanup(ctx context.Context) {
15     mustExec(ctx, s.pool, 'DROP INDEX IF EXISTS idx_bench_k;')
16 }

```

Листинг 4.3 — Сценарий UNIQUE B-Tree по атрибуту k

```

1 type heapUniqueBtree struct{ pool *pgxpool.Pool }
2
3 func (s heapUniqueBtree) Name() string { return
    "heap_unique_btree" }
4 func (s heapUniqueBtree) Prepare(ctx context.Context) {
5     mustExec(ctx, s.pool, 'DROP INDEX IF EXISTS idx_bench_k;')
6 }
7 func (s heapUniqueBtree) Reset(ctx context.Context) {
8     mustExec(ctx, s.pool, 'DROP INDEX IF EXISTS idx_bench_k;')
9     mustExec(ctx, s.pool, 'CREATE UNIQUE INDEX idx_bench_k ON
    bench_items(k);')

```

```

10 }
11 func (s heapUniqueBtree) PostInsertLayout(ctx context.Context) {
12     mustExec(ctx, s.pool, 'ANALYZE bench_items;')
13 }
14 func (s heapUniqueBtree) Cleanup(ctx context.Context) {
15     mustExec(ctx, s.pool, 'DROP INDEX IF EXISTS idx_bench_k;')
16 }

```

Листинг 4.4 — Сценарий с укладкой CLUSTER по k

```

1 type clusteredBtree struct{ pool *pgxpool.Pool }
2
3 func (s clusteredBtree) Name() string { return "clustered_btree" }
4 func (s clusteredBtree) Prepare(ctx context.Context) {
5     mustExec(ctx, s.pool, 'DROP INDEX IF EXISTS idx_bench_k;')
6 }
7 func (s clusteredBtree) Reset(ctx context.Context) {
8     mustExec(ctx, s.pool, 'DROP INDEX IF EXISTS idx_bench_k;')
9     mustExec(ctx, s.pool, 'CREATE INDEX idx_bench_k ON
10         bench_items(k);')
11 }
12 func (s clusteredBtree) PostInsertLayout(ctx context.Context) {
13     mustExec(ctx, s.pool, 'CLUSTER bench_items USING
14         idx_bench_k;')
15     mustExec(ctx, s.pool, 'ANALYZE bench_items;')
16 }
17 func (s clusteredBtree) Cleanup(ctx context.Context) {
18     mustExec(ctx, s.pool, 'DROP INDEX IF EXISTS idx_bench_k;')
19 }

```

4.3 Результаты замеров

В таблицах 4.1–4.3 представлены результаты измерения медианного времени (мс) от объёма N для трёх операций и четырёх сценариев. Для каждого сценария и объёма операций было выполнено 100 замеров.

На рисунках 4.1–4.3 представлена визуализация результатов:

Таблица 4.1 — Время выполнения операции удаления, в мс

Сценарий n	кластер	некластер	без индекса	уник некластер
10000	102.77	105.50	59.07	103.61
40000	312.88	312.83	189.67	314.87
70000	532.92	528.74	318.15	535.05
100000	749.52	763.72	447.66	759.35
130000	953.86	1002.66	586.30	978.92
160000	1170.91	1233.65	716.02	1207.56
190000	1413.88	1465.53	843.07	1420.78

Таблица 4.2 — Время выполнения операции вставки, в мс

Сценарий n	кластер	некластер	без индекса	уник некластер
10000	99.06	76.99	57.14	99.46
40000	306.83	306.55	184.22	307.66
70000	518.94	523.89	310.42	526.15
100000	734.69	754.65	435.56	742.64
130000	973.08	981.66	572.50	958.72
160000	1155.45	1221.09	699.15	1184.32
190000	1395.56	1443.21	822.09	1404.70

Таблица 4.3 — Время выполнения операции чтения, в мс

Сценарий n	кластер	некластер	без индекса	уник некластер
10000	0.19	0.18	0.59	0.20
40000	0.26	0.34	1.89	0.36
70000	0.28	0.52	3.32	0.45
100000	0.28	0.53	4.67	0.51
130000	0.30	0.56	5.71	0.57
160000	0.27	0.58	7.05	0.56
190000	0.28	0.62	7.81	0.59

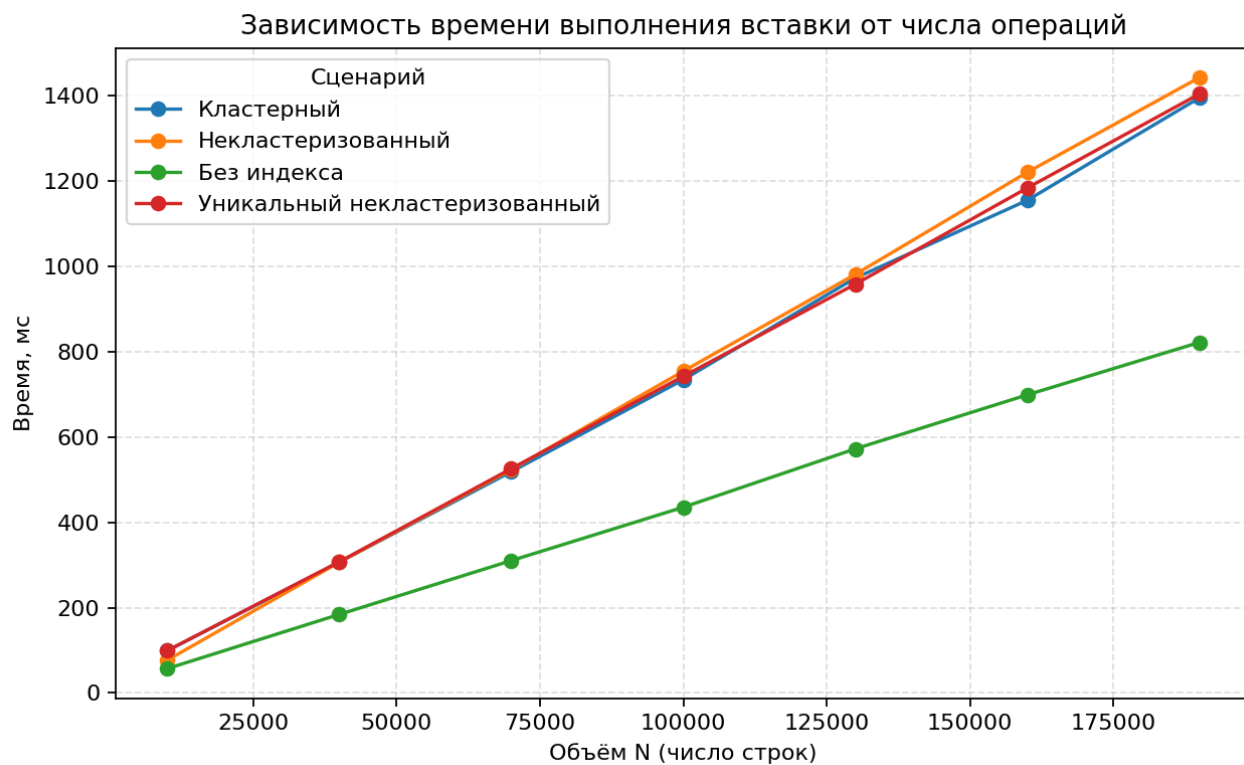


Рисунок 4.1 — Зависимость времени от числа операций вставки.

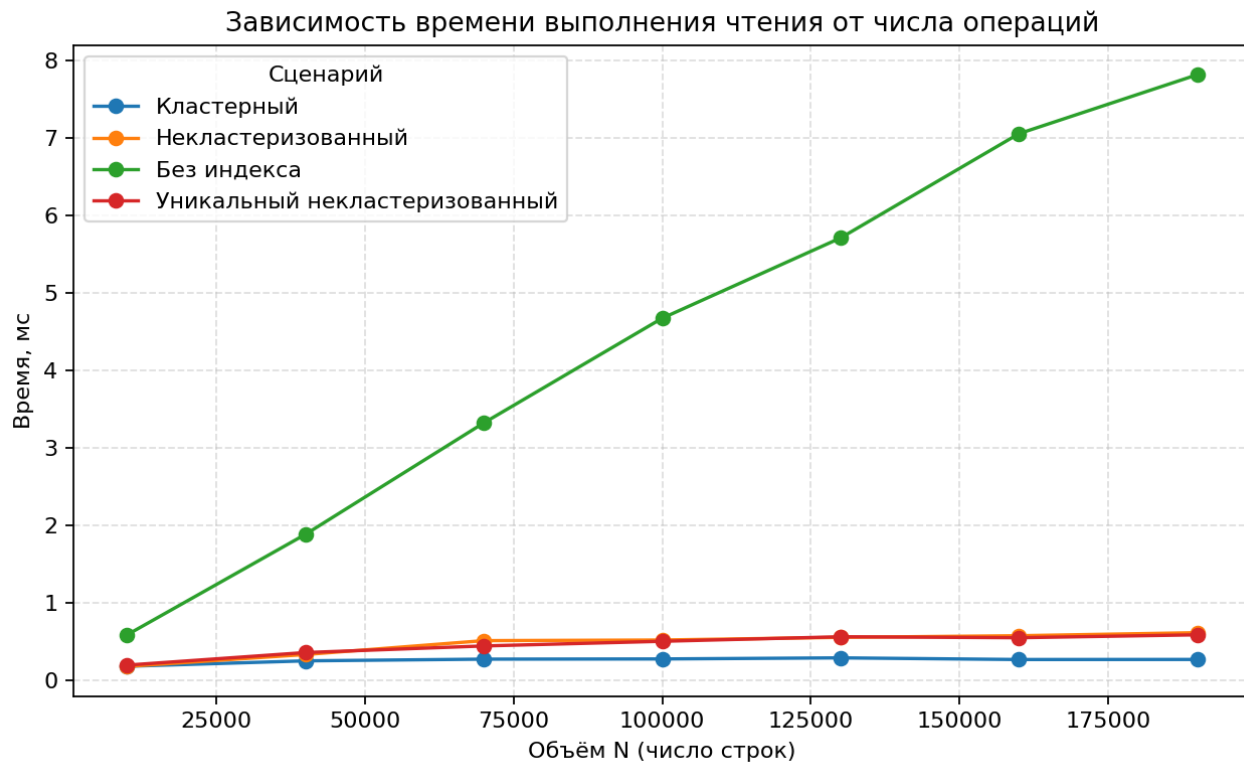


Рисунок 4.2 — Зависимость времени от числа операций выбора.

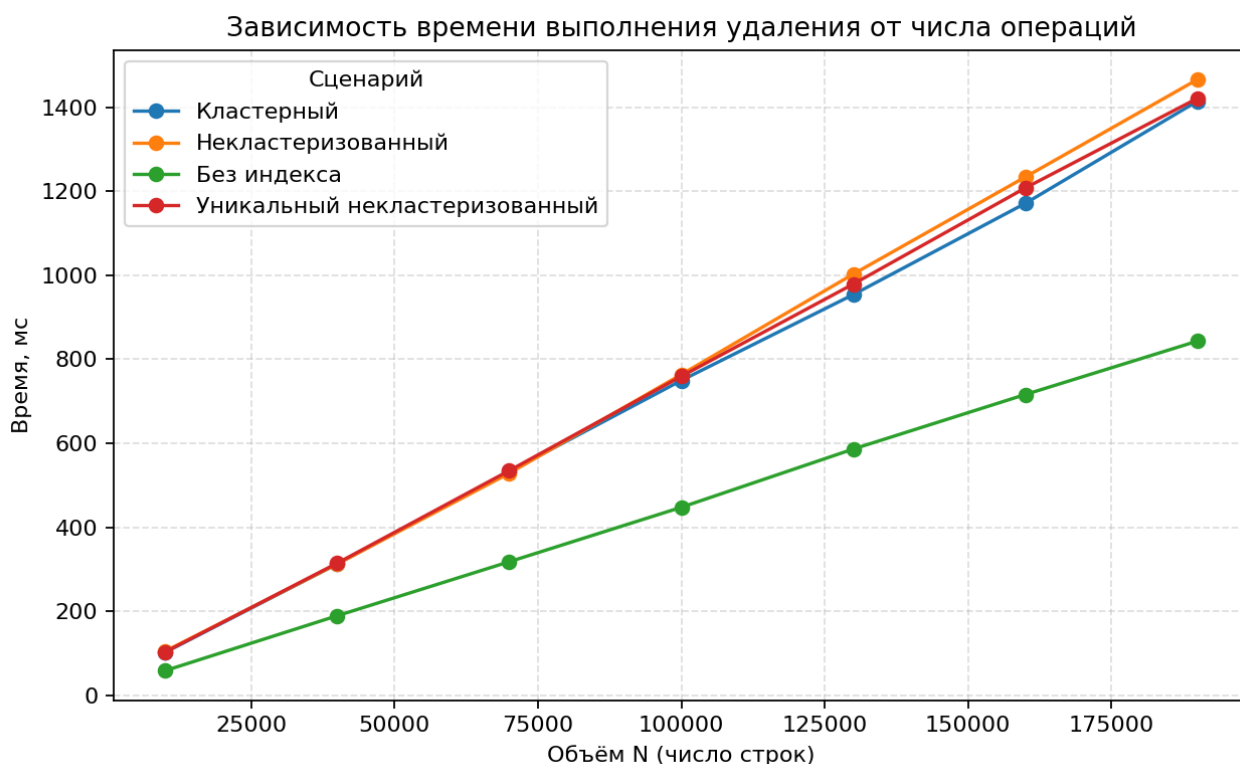


Рисунок 4.3 — Зависимость времени от числа операций удаления.

Вывод

В данном разделе описаны технические характеристики ЭВМ, на которой проводилось исследование, а также представлены ход выполнения исследования и его результаты. Полученные данные подтвердили ожидания: Время выполнения различных операций для разных типов индексов отличается.

Операция вставки выполняется быстрее примерно на 70% при больших объемах данных, по сравнению с остальными сценариями. Это объясняется тем, что без индекса записи просто добавляются в кучу, тогда как для некластерных и уникальных индексов требуется дополнительное обновление бинарного дерева. Уникальный индекс работает медленнее обычного, так как дополнительно проверяет уникальность ключей.

Аналогично, быстрее всего удаление происходит без индекса, примерно на 65% по сравнению с остальными вариантами. При наличии индекса, кроме пометки строк в куче, требуется удаление соответствующих ссылок из дерева, что и увеличивает время. Разница между кластерным и некластерным индексами незначительна.

Наиболее заметный эффект индексы дают операции чтения: время выпол-

нения с индексами остаётся почти постоянным даже при росте N , тогда как без индекса линейно увеличивается. Кластеризация немного улучшает результаты по сравнению с индексами без кластеризации, благодаря упорядоченному физическому размещению строк. Уникальный индекс по скорости чтения сопоставим с обычным.

ЗАКЛЮЧЕНИЕ

Цель данной работы была достигнута: разработана база данных для хранения наборов данных (датасетов), также создано приложения для доступа к ней. Все поставленные задачи были успешно решены:

- проведён анализ предметной области и выделены ключевые сущности, а также их взаимосвязи;
- сформулированы требования к системе, включая поддержку ролевой модели и обеспечение целостности данных;
- спроектирована и реализованна база данных с ограничениями и триггером;
- разработаны и реализованы основные компоненты приложения, взаимодействующего с базой данных;
- проведено исследование влияния различных типов индексов на производительность запросов.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Гульчеев В. А. Секреты датасетов: практическое руководство по анализу и обработке данных. Самиздат, 2023. — С. 42.
2. Kaggle datasets [Электронный ресурс]. Режим доступа: <https://www.kaggle.com/datasets/> (Дата обращения: 16.03.2025).
3. Hugging face datasets [Электронный ресурс]. Режим доступа: <https://huggingface.co/docs/datasets/index> (Дата обращения: 17.03.2025).
4. ClearML datasets [Электронный ресурс]. Режим доступа: <https://clear.ml/docs/latest/docs/hyperdatasets/dataset/> (Дата обращения: 17.03.2025).
5. Чехарин Е. Е. Big data big problems. International Scientific Electronic Journal, 2016. — С. 5.
6. UCI Machine Learning Repository [Электронный ресурс]. Режим доступа: <https://archive.ics.uci.edu/> (Дата обращения: 17.03.2025).
7. Кренке Д. Теория и практика построения баз данных. СПб. Питер, 2003. — С. 800.
8. Гущин А. Н. Базы данных. 2-е изд. М. Берлин: Директ-Медиа, 2015. — С. 311.
9. Ю. Ю. Громов О. Г. Иванова. Управление данными. Тамбов. Издательство ФГБОУ ВПО «ТГТУ», 2015. — С. 193.
10. Васильева К. Н. Хусаинова Г. Я. Реляционные базы данных. Colloquium-journal, 2020. — С. 23.
11. Appropriate Uses For SQLite [Электронный ресурс]. Режим доступа: <https://sqlite.org/whentouse.html> (Дата обращения: 25.04.2025).
12. MySQL documentation [Электронный ресурс]. Режим доступа: <https://dev.mysql.com/doc/> (Дата обращения: 25.04.2025).

13. PostgreSQL documentation [Электронный ресурс]. Режим доступа: <https://www.postgresql.org/docs/> (Дата обращения: 25.04.2025).
14. S3 documentation [Электронный ресурс]. Режим доступа: <https://dev.mysql.com/doc/> (Дата обращения: 25.04.2025).
15. Go documentation [Электронный ресурс]. Режим доступа: <https://go.dev/doc/> (Дата обращения: 25.04.2025).
16. AMD Ryzen™ 7–5800H [Электронный ресурс]. Режим доступа: <https://www.amd.com/en/support/downloads/drivers.html/processors/ryzen/ryzen-5000-series/amd-ryzen-7-5800h.html> (Дата обращения: 25.04.2025).
17. Ubuntu 24.04 [Электронный ресурс]. Режим доступа: <https://ubuntu.com/blog/tag/ubuntu-24-04-lts> (Дата обращения: 25.04.2025).

ПРИЛОЖЕНИЕ А

Презентация к курсовой работе

Презентация содержит 10 слайдов.